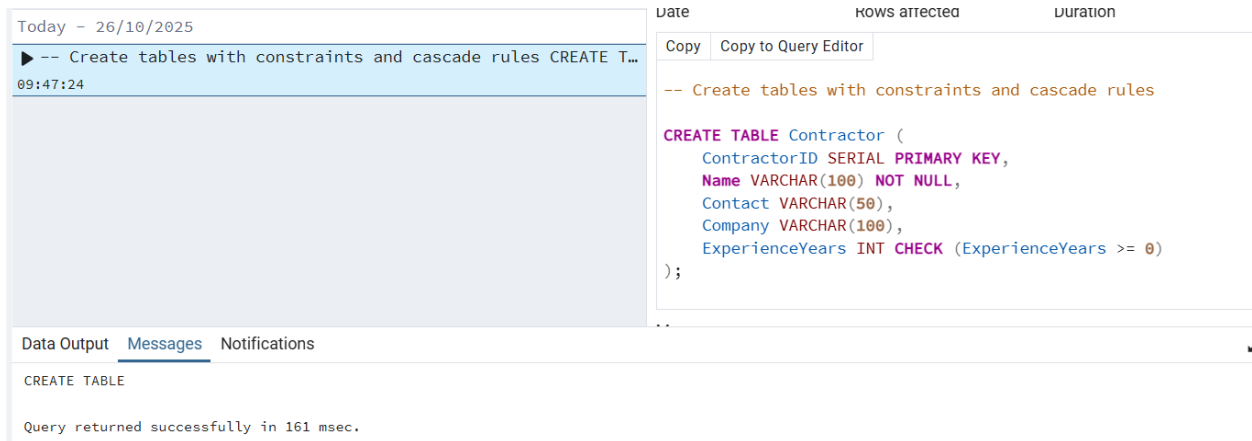


1. Create all six tables

i. Table Contractor

-- Create tables with constraints and cascade rules

```
CREATE TABLE Contractor (  
  
    ContractorID SERIAL PRIMARY KEY,  
  
    Name VARCHAR(100) NOT NULL,  
  
    Contact VARCHAR(50),  
  
    Company VARCHAR(100),  
  
    ExperienceYears INT CHECK (ExperienceYears >= 0)  
  
);
```



ii. Table Project

```
CREATE TABLE Project (  
  
    ProjectID SERIAL PRIMARY KEY,  
  
    ContractorID INT REFERENCES Contractor(ContractorID) ON DELETE CASCADE,  
  
    Name VARCHAR(100) NOT NULL,  
  
    Location VARCHAR(100),
```

```

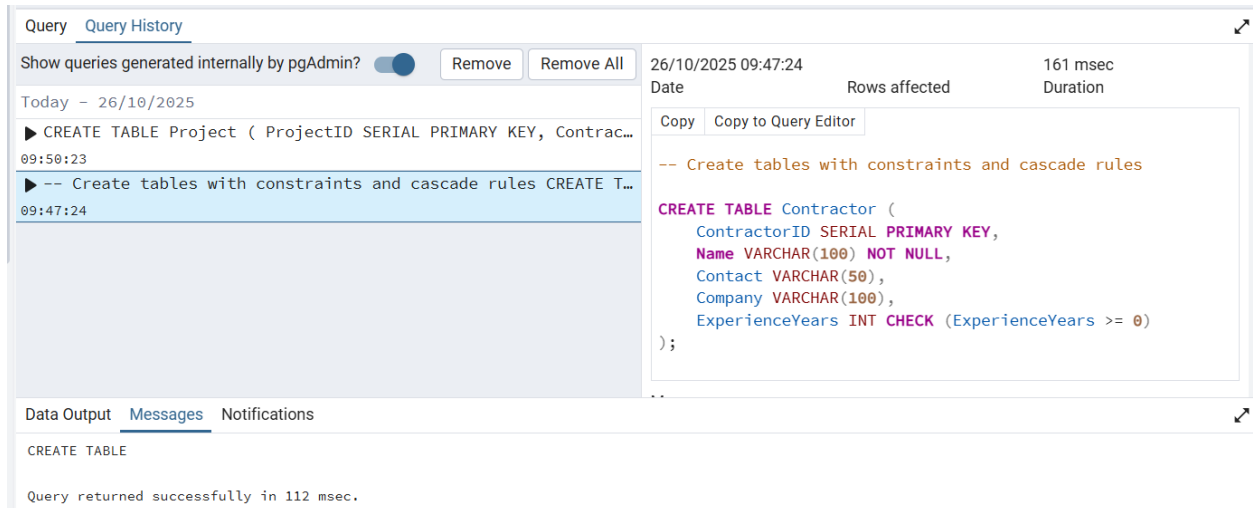
StartDate DATE,

EndDate DATE,

Budget NUMERIC(12,2) CHECK (Budget > 0)

);

```



iii. Table material

```

CREATE TABLE Material (

MaterialID SERIAL PRIMARY KEY,

ProjectID INT REFERENCES Project(ProjectID) ON DELETE CASCADE,

MaterialName VARCHAR(100) NOT NULL,

Cost NUMERIC(10,2) CHECK (Cost >= 0),

Quantity INT CHECK (Quantity >= 0),

Supplier VARCHAR(100)

);

```

Query [Query History](#)

Show queries generated internally by pgAdmin? ☒ Remove Remove All

Today - 26/10/2025

- ▶ CREATE TABLE Material (MaterialID SERIAL PRIMARY KEY, Proje... 09:51:58
- ▶ CREATE TABLE Project (ProjectID SERIAL PRIMARY KEY, Contrac... 09:50:23
- ▶ -- Create tables with constraints and cascade rules CREATE T... 09:47:24

26/10/2025 09:47:24 161 msec

Date Rows affected Duration

Copy Copy to Query Editor

```
-- Create tables with constraints and cascade rules

CREATE TABLE Contractor (
    ContractorID SERIAL PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Contact VARCHAR(50),
    Company VARCHAR(100),
    ExperienceYears INT CHECK (ExperienceYears >= 0)
);
```

Data Output [Messages](#) Notifications

CREATE TABLE

Query returned successfully in 154 msec.

iv. Table Equipment

```
CREATE TABLE Equipment (
    EquipmentID SERIAL PRIMARY KEY,
    ProjectID INT REFERENCES Project(ProjectID) ON DELETE CASCADE,
    Name VARCHAR(100) NOT NULL,
    CostPerDay NUMERIC(10,2) CHECK (CostPerDay >= 0),
    Status VARCHAR(50)
);
```

Query [Query History](#)

Show queries generated internally by pgAdmin? ☒ Remove Remove All

Today - 26/10/2025

- ▶ CREATE TABLE Equipment (EquipmentID SERIAL PRIMARY KEY, Pro... 09:54:49
- ▶ CREATE TABLE Material (MaterialID SERIAL PRIMARY KEY, Proje... 09:51:58
- ▶ CREATE TABLE Project (ProjectID SERIAL PRIMARY KEY, Contrac... 09:50:23
- ▶ -- Create tables with constraints and cascade rules CREATE T... 09:47:24

26/10/2025 09:47:24 161 msec

Date Rows affected Duration

Copy Copy to Query Editor

```
-- Create tables with constraints and cascade rules

CREATE TABLE Contractor (
    ContractorID SERIAL PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Contact VARCHAR(50),
    Company VARCHAR(100),
    ExperienceYears INT CHECK (ExperienceYears >= 0)
);
```

Data Output [Messages](#) Notifications

CREATE TABLE

Query returned successfully in 121 msec.

v. Table: Expense

```
CREATE TABLE Expense (  
  
    ExpenseID SERIAL PRIMARY KEY,  
  
    ProjectID INT REFERENCES Project(ProjectID) ON DELETE CASCADE,  
  
    Description TEXT,  
  
    Amount NUMERIC(10,2) CHECK (Amount >= 0),  
  
    Date DATE  
  
);
```

The screenshot shows the pgAdmin interface with the 'Query' tab selected. The 'Query History' pane on the left lists several queries, with the most recent one highlighted: '-- Create tables with constraints and cascade rules CREATE T...'. The main pane displays the SQL query for creating the 'Contractor' table, which is part of a larger script. The query is:

```
CREATE TABLE Contractor (  
    ContractorID SERIAL PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Contact VARCHAR(50),  
    Company VARCHAR(100),  
    ExperienceYears INT CHECK (ExperienceYears >= 0)  
);
```

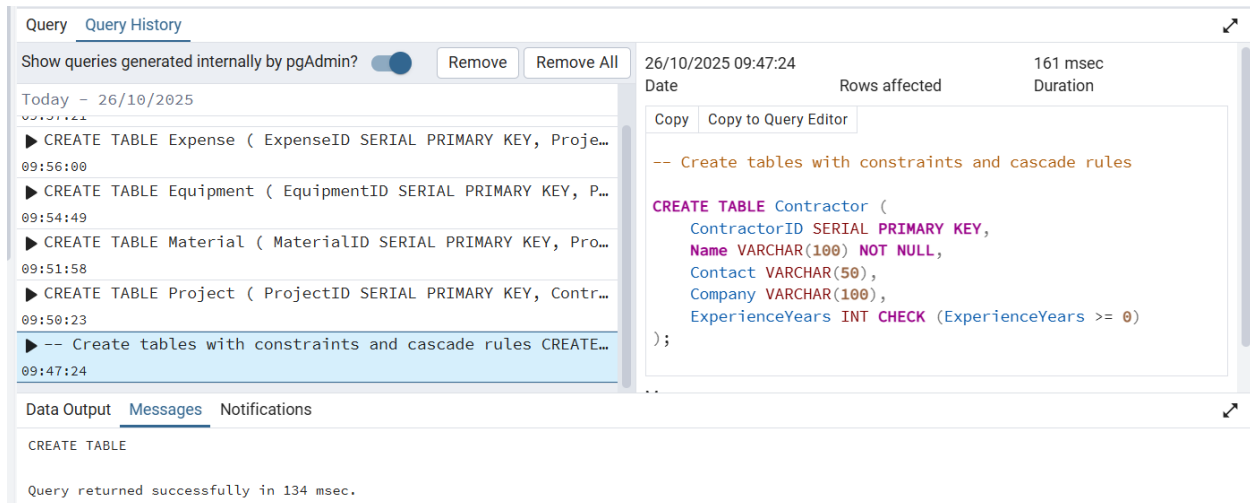

The 'Data Output' pane at the bottom shows the message: 'Query returned successfully in 100 msec.'

vi. Table: Payment

```
CREATE TABLE Payment (  
  
    PaymentID SERIAL PRIMARY KEY,  
  
    ContractorID INT REFERENCES Contractor(ContractorID) ON DELETE CASCADE,  
  
    ProjectID INT REFERENCES Project(ProjectID),  
  
    Amount NUMERIC(10,2) CHECK (Amount >= 0),  
  
    PaymentDate DATE,
```

Method VARCHAR(50)

);



2. I used DELETE CASCADE inside table creation query where foreign key is enforced.

3. Insert records

i. Into table contractor

INSERT INTO contractor (name, contact, company, experienceyears)

VALUES

('Main contractors / consortium', '0730520520', 'Urbacon Trading & Contracting (UCC) / UCC Holding (Qatar)', 20),

('Contractor awarded', '0759229743', 'China Railway Construction Engineering Group Co., Ltd. (CRCEG)', 25),

('AfDB-backed programme', '078885643', 'Rwanda Urban Mobility Improvement (RUMI)', 15),

('UCC Holding (UCC / Urbacon)', '0788567843', 'CCC (consortium)', 10),

('Equity Bank Group', '075678900', 'Equity Holding Venture Capital', 6);

Show queries generated internally by pgAdmin? ☒ Remove Remove All

Today - 26/10/2025

- ▶ INSERT INTO contractor (name, contact, company, experience... 10:30:47
- ▶ CREATE TABLE Payment (PaymentID SERIAL PRIMARY KEY, Contr... 09:57:21
- ▶ CREATE TABLE Expense (ExpenseID SERIAL PRIMARY KEY, Proje... 09:56:00
- ▶ CREATE TABLE Equipment (EquipmentID SERIAL PRIMARY KEY, P... 09:54:49
- ▶ CREATE TABLE Material (MaterialID SERIAL PRIMARY KEY, Pro... 09:51:58
- ▶ CREATE TABLE Project (ProjectID SERIAL PRIMARY KEY, Contr...

Data Output Messages Notifications

INSERT 0 5

Query returned successfully in 485 msec.

26/10/2025 10:30:47 485 msec

Date Rows affected Duration

Copy Copy to Query Editor

```
INSERT INTO contractor (name, contact, company, experienceyea
VALUES
('Main contractors / consortium', '0730520520', 'Urbakon Trad
('Contractor awarded', '0759229743', 'China Railway Construct
('AfDB-backed programme', '078885643', 'Rwanda Urban Mobility
('UCC Holding (UCC / Urbakon)', '0788567843', 'CCC (consortiu
('Equity Bank Group', '075678900', 'Equity Holding Venture Ca
```

Messages

Query returned successfully in 485 msec.

ii. Into table Project

INSERT INTO Project (ContractorID, Name, Location, StartDate, EndDate, Budget)

VALUES

(1, 'Kacyiru Office Complex', 'Kigali', '2025-01-15', '2025-12-20', 95000000),

(1, 'Gisenyi Waterfront Hotel', 'Rubavu', '2025-03-10', '2026-02-28', 72000000),

(3, 'Kigali Convention Parking', 'Gasabo', '2025-05-01', '2025-11-15', 54000000),

(2, 'Butare University Dorms', 'Huye', '2025-02-01', '2025-09-30', 30000000),

(3, 'Musanze Housing Project', 'Musanze', '2025-04-12', '2026-01-31', 87500000);

Show queries generated internally by pgAdmin? ☒ Remove Remove All

Today - 26/10/2025

- ▶ INSERT INTO Project (ContractorID, Name, Location, StartDa... 10:38:08
- ▶ INSERT INTO contractor (name, contact, company, experience... 10:30:47
- ▶ CREATE TABLE Payment (PaymentID SERIAL PRIMARY KEY, Contr... 09:57:21
- ▶ CREATE TABLE Expense (ExpenseID SERIAL PRIMARY KEY, Proje... 09:56:00
- ▶ CREATE TABLE Equipment (EquipmentID SERIAL PRIMARY KEY, P... 09:54:49
- ▶ CREATE TABLE Material (MaterialID SERIAL PRIMARY KEY, Pro...

Data Output Messages Notifications

INSERT 0 5

Query returned successfully in 249 msec.

26/10/2025 10:38:08 249 msec

Date Rows affected Duration

Copy Copy to Query Editor

```
INSERT INTO Project (ContractorID, Name, Location, StartDate,
VALUES
(1, 'Kacyiru Office Complex', 'Kigali', '2025-01-15', '2025-1
(1, 'Gisenyi Waterfront Hotel', 'Rubavu', '2025-03-10', '2026
(3, 'Kigali Convention Parking', 'Gasabo', '2025-05-01', '202
(2, 'Butare University Dorms', 'Huye', '2025-02-01', '2025-09
(3, 'Musanze Housing Project', 'Musanze', '2025-04-12', '2026
```

Messages

iii. Into table Expense

INSERT INTO Expense (ProjectID, Description, Amount, Date)

VALUES

(1, 'Foundation excavation and concrete work', 15000000, '2025-02-10'),

(2, 'Hotel interior finishing supplies', 12000000, '2025-03-15'),

(3, 'Underground parking lighting installation', 8000000, '2025-06-05'),

(4, 'Dormitory furniture procurement', 7000000, '2025-04-20'),

(5, 'Housing site clearance and design fees', 10000000, '2025-02-25');

The screenshot displays the pgAdmin interface. On the left, the 'Query History' tab is active, showing a list of queries. The first query, 'INSERT INTO Expense (ProjectID, Description, Amount, Date)...', is highlighted. The right pane shows the SQL query and its execution results. The query is:
`INSERT INTO Expense (ProjectID, Description, Amount, Date)`
`VALUES`
`(1, 'Foundation excavation and concrete work', 15000000, '2025-02-10'),`
`(2, 'Hotel interior finishing supplies', 12000000, '2025-03-15'),`
`(3, 'Underground parking lighting installation', 8000000, '2025-06-05'),`
`(4, 'Dormitory furniture procurement', 7000000, '2025-04-20')`
`(5, 'Housing site clearance and design fees', 10000000, '2025-02-25');`
The execution results show: Date: 26/10/2025 10:43:29, Rows affected: 5, Duration: 182 msec. At the bottom, a message states: 'Query returned successfully in 182 msec.'

4. Retrieve all expenses per project.

SELECT ProjectID, SUM(Amount) AS TotalExpense

FROM Expense GROUP BY ProjectID;

	projectid integer	totalexpanse numeric
1	3	8000000.00
2	5	10000000.00
3	4	7000000.00
4	2	12000000.00
5	1	15000000.00

Total rows: 5
Query complete 00:00:00.219

5. Update equipment status after repair.

update equipment set status = 'Repaired';

Query
Query History

Show queries generated internally by pgAdmin? ☒ Remove Remove All

Today - 26/10/2025

▶ update equipment set status = 'Repaired';
10:57:10
▶ SELECT ProjectID, SUM(Amount) AS TotalExpense FROM Expens...
10:51:44
▶ SELECT ProjectID, SUM(Amount) AS TotalExpense FROM Expens...
10:51:24
▶ INSERT INTO Expense (ProjectID, Description, Amount, Date)...
10:43:29
▶ INSERT INTO Project (ContractorID, Name, Location, StartDa...
10:38:08
▶ INSERT INTO contractor (name, contact, company, experience...

Data Output
Messages
Notifications

UPDATE 0

Query returned successfully in 170 msec.

26/10/2025 10:57:10
Date
Rows affected
170 msec
Duration
Copy Copy to Query Editor

update equipment set status = 'Repaired';

Messages
Query returned successfully in 170 msec.

6. Identify projects exceeding 80% of their budget.

```
SELECT

    p.ProjectID,

    p.Name AS ProjectName,

    p.Budget,

    SUM(e.Amount) AS TotalExpense,

    ROUND((SUM(e.Amount) / p.Budget) * 100, 2) AS PercentUsed

FROM Project p

JOIN Expense e ON p.ProjectID = e.ProjectID

GROUP BY p.ProjectID, p.Name, p.Budget

HAVING SUM(e.Amount) > 0.8 * p.Budget;
```

The screenshot shows a database query editor interface. The top bar includes tabs for 'Query' and 'Query History'. Below this, there's a toggle for 'Show queries generated internally by pgAdmin?' and buttons for 'Remove' and 'Remove All'. The main area displays a list of queries on the left and the selected query on the right. The selected query is the SQL query for identifying projects exceeding 80% of their budget. The bottom of the interface shows a table structure with columns: projectid (integer), projectname (character varying (100)), budget (numeric (12,2)), totalexpense (numeric), and percentused (numeric).

```
Query Query History

Show queries generated internally by pgAdmin? ☒ Remove Remove All

Today - 26/10/2025

▶ SELECT p.ProjectID, p.Name AS ProjectName, p.Budget, SUM(e...
11:00:33
▶ update equipment set status = 'Repaired';
10:57:10
▶ SELECT ProjectID, SUM(Amount) AS TotalExpense FROM Expens...
10:51:44
▶ SELECT ProjectID, SUM(Amount) AS TotalExpense FROM Expens...
10:51:24
▶ INSERT INTO Expense (ProjectID, Description, Amount, Date)...
10:43:29
▶ INSERT INTO Project (ContractorID, Name, Location, StartDa...
Data Output Messages Notifications

SELECT
    p.ProjectID,
    p.Name AS ProjectName,
    p.Budget,
    SUM(e.Amount) AS TotalExpense,
    ROUND((SUM(e.Amount) / p.Budget) * 100, 2) AS PercentUsed
FROM Project p
JOIN Expense e ON p.ProjectID = e.ProjectID
GROUP BY p.ProjectID, p.Name, p.Budget
HAVING SUM(e.Amount) > 0.8 * p.Budget;
```

projectid	projectname	budget	totalexpense	percentused
[PK] integer	character varying (100)	numeric (12,2)	numeric	numeric

7. Create a view showing total payments by contractor.

```
SELECT

    p.ProjectID,

    p.Name AS ProjectName,

    p.Budget,
```

```

SUM(e.Amount) AS TotalExpense,

ROUND((SUM(e.Amount) / p.Budget) * 100, 2) AS PercentUsed

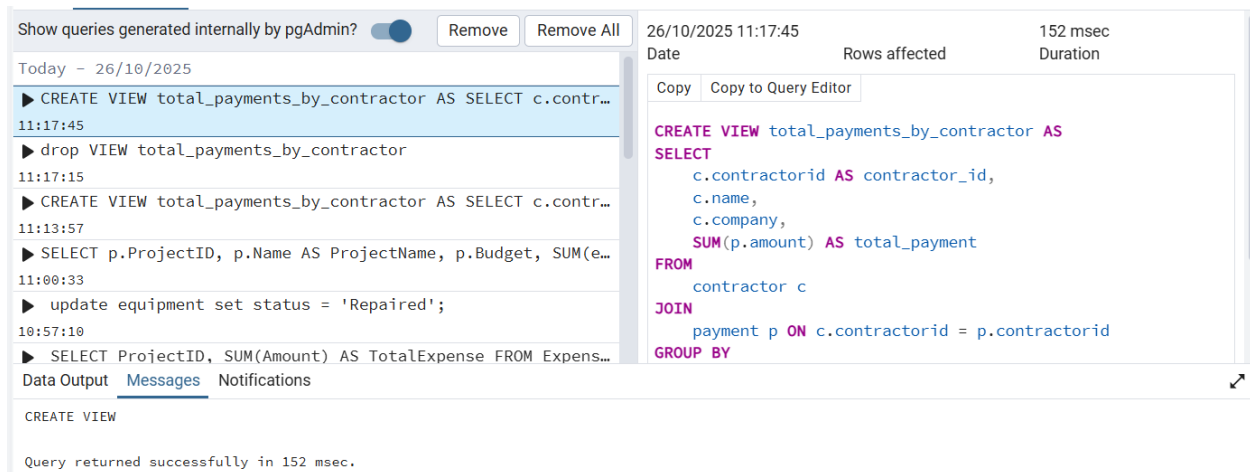
FROM Project p

JOIN Expense e ON p.ProjectID = e.ProjectID

GROUP BY p.ProjectID, p.Name, p.Budget

HAVING SUM(e.Amount) > 0.8 * p.Budget;

```



The screenshot shows the pgAdmin interface. On the left, a list of queries is displayed with timestamps. The selected query is: `SELECT ProjectID, SUM(Amount) AS TotalExpense FROM Expense...`. On the right, the SQL editor shows the following query: `CREATE VIEW total_payments_by_contractor AS SELECT c.contractorid AS contractor_id, c.name, c.company, SUM(p.amount) AS total_payment FROM contractor c JOIN payment p ON c.contractorid = p.contractorid GROUP BY`. The execution results on the right show the date 26/10/2025 11:17:45, 152 msec duration, and a message: "Query returned successfully in 152 msec."

8. Implement a trigger that prevents new expenses if total exceeds project budget.

```
CREATE OR REPLACE FUNCTION prevent_budget_overrun()
```

```
RETURNS TRIGGER AS $$
```

```
DECLARE
```

```
total_expenses NUMERIC(12,2);
```

```
project_budget NUMERIC(12,2);
```

```
BEGIN
```

```
-- Get current total expenses for the project
```

```
SELECT COALESCE(SUM(amount), 0) INTO total_expenses
```

```
FROM expense
```

```

WHERE project_id = NEW.project_id;

-- Get the project budget

SELECT budget INTO project_budget

FROM project

WHERE id = NEW.project_id;

-- Check if adding the new expense exceeds the budget

IF (total_expenses + NEW.amount) > project_budget THEN

    RAISE EXCEPTION

    'Cannot insert expense: total (%.2f) exceeds project budget (%.2f)',

    total_expenses + NEW.amount, project_budget;

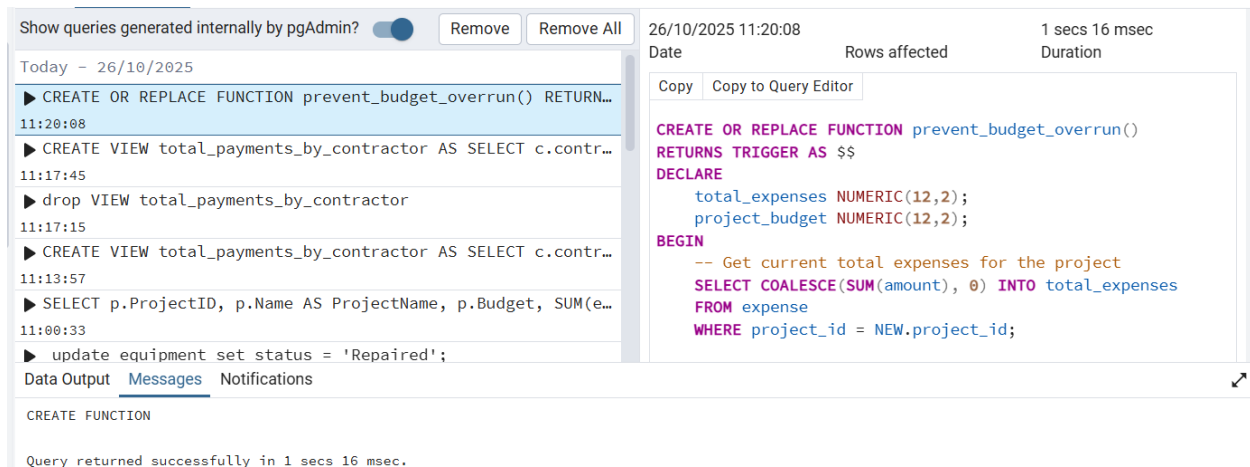
END IF;

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

```



The screenshot shows a PostgreSQL query editor interface. On the left, there is a list of queries executed today (26/10/2025). The selected query is:

```

CREATE OR REPLACE FUNCTION prevent_budget_overrun() RETURN...
11:20:08
CREATE VIEW total_payments_by_contractor AS SELECT c.contr...
11:17:45
drop VIEW total_payments_by_contractor
11:17:15
CREATE VIEW total_payments_by_contractor AS SELECT c.contr...
11:13:57
SELECT p.ProjectID, p.Name AS ProjectName, p.Budget, SUM(e...
11:00:33
update equipment set status = 'Repaired';

```

On the right, the details of the selected query are shown, including the date (26/10/2025 11:20:08), rows affected, and duration (1 secs 16 msec). The query text is:

```

CREATE OR REPLACE FUNCTION prevent_budget_overrun()
RETURNS TRIGGER AS $$
DECLARE
    total_expenses NUMERIC(12,2);
    project_budget NUMERIC(12,2);
BEGIN
    -- Get current total expenses for the project
    SELECT COALESCE(SUM(amount), 0) INTO total_expenses
    FROM expense
    WHERE project_id = NEW.project_id;

```

At the bottom, a status message indicates: "Query returned successfully in 1 secs 16 msec."

```

CREATE TRIGGER check_project_budget

BEFORE INSERT ON expense

```

FOR EACH ROW

EXECUTE FUNCTION prevent_budget_overrun());

The screenshot shows the pgAdmin 4 interface. On the left, a list of queries is displayed with timestamps. The main pane shows the execution of a SQL query. The query text is: `CREATE TRIGGER check_project_budget BEFORE INSERT ON expense FOR EACH ROW EXECUTE FUNCTION prevent_budget_overrun();`. The execution status is 'Query returned successfully in 95 msec.'.

Date	Rows affected	Duration
26/10/2025 11:22:13		95 msec

Parallel and Distributed Databases

Practical Lab Assessment Sheet

Pre-requisites

- Create first the roles, let's call them branchA and btanchB with password "labpass"
 - CREATE ROLE brancha LOGIN PASSWORD 'labpass';
 - CREATE ROLE branchb LOGIN PASSWORD 'labpass';
- Create schema:
 - CREATE SCHEMA branchA AUTHORIZATION branchA;
 - CREATE SCHEMA branchb AUTHORIZATION branchb;

- Grant options

1. Distributed Schema Design and Fragmentation

BranchA

SET search_path TO brancha;

CREATE TABLE project (

id SERIAL PRIMARY KEY,

name VARCHAR(100),

```

        budget NUMERIC(14,2)
    );

CREATE TABLE expense (
    id SERIAL PRIMARY KEY,
    project_id INT NOT NULL REFERENCES project(id),
    amount NUMERIC(14,2),
    description VARCHAR(200),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE transactions (
    tx_id SERIAL PRIMARY KEY,
    project_id INT REFERENCES project(id),
    tx_date DATE,
    amount NUMERIC(14,2)
);

-- Insert a sample record
INSERT INTO project(name, budget) VALUES ('Road Construction', 1000000);

COMMIT;

```

Show queries generated internally by pgAdmin? ☒ Remove Remove All
26/10/2025 14:23:23
Date
Rows affected
1 secs 350 msec
Duration

Today - 26/10/2025

▶ SET search_path TO brancha; CREATE TABLE project (id SERIAL...
14:23:23
▶ SET search_path TO brancha;
14:22:46
▶ CONNECT BRANCHA/labpass
12:08:04
▶ CONNECT BRANCHA/labpass CREATE TABLE project (id NUMBER G...
12:05:49
▶ CREATE SCHEMA branchb AUTHORIZATION branchb;
12:05:11
▶ GRANT CONNECT, RESOURCE, CREATE SESSION, CREATE TABLE, CRE...

Copy Copy to Query Editor

```

SET search_path TO brancha;
CREATE TABLE project (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  budget NUMERIC(14,2)
);

CREATE TABLE expense (
  id SERIAL PRIMARY KEY,
  project_id INT NOT NULL REFERENCES project(id),
  amount NUMERIC(14,2),

```

Data Output Messages Notifications

WARNING: there is no transaction in progress
COMMIT
Query returned successfully in 1 secs 350 msec.

BranchB

SET search_path TO branchb;

CREATE TABLE project (

id SERIAL PRIMARY KEY,

name VARCHAR(100),

budget NUMERIC(14,2)

);

CREATE TABLE expense (

id SERIAL PRIMARY KEY,

project_id INT NOT NULL REFERENCES project(id),

amount NUMERIC(14,2),

description VARCHAR(200),

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE transactions (

tx_id SERIAL PRIMARY KEY,

```
project_id INT REFERENCES project(id),  
  
tx_date DATE,  
  
amount NUMERIC(14,2)  
  
);
```

The screenshot shows a database query editor with a toolbar at the top containing icons for file operations, editing, and execution. Below the toolbar are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL script with line numbers 1 through 16. The script defines two tables: 'project' and 'expense'. The 'project' table has columns 'id' (SERIAL PRIMARY KEY), 'name' (VARCHAR(100)), and 'budget' (NUMERIC(14,2)). The 'expense' table has columns 'id' (SERIAL PRIMARY KEY), 'project_id' (INT NOT NULL REFERENCES project(id)), 'amount' (NUMERIC(14,2)), 'description' (VARCHAR(200)), and 'created_at' (TIMESTAMP DEFAULT CURRENT_TIMESTAMP). Below the query editor are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the text 'CREATE TABLE' and a confirmation message: 'Query returned successfully in 315 msec.'

```
1 SET search_path TO branchb;  
2  
3 CREATE TABLE project (  
4     id SERIAL PRIMARY KEY,  
5     name VARCHAR(100),  
6     budget NUMERIC(14,2)  
7 );  
8  
9 CREATE TABLE expense (  
10    id SERIAL PRIMARY KEY,  
11    project_id INT NOT NULL REFERENCES project(id),  
12    amount NUMERIC(14,2),  
13    description VARCHAR(200),  
14    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
15 );  
16
```

Data Output **Messages** Notifications

CREATE TABLE

Query returned successfully in 315 msec.

2. Create and Use Database Links

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;
```

Data Output Messages Notifications

CREATE EXTENSION

Query returned successfully in 1 secs 86 msec.

CREATE SERVER branch_server

FOREIGN DATA WRAPPER postgres_fdw

OPTIONS (host 'localhost', dbname 'branchdb', port '5432');

Data Output Messages Notifications

CREATE SERVER

Query returned successfully in 185 msec.

CREATE USER MAPPING FOR brancha

SERVER branch_server

OPTIONS (user 'brancha', password 'labpass');

Data Output Messages Notifications

CREATE USER MAPPING

Query returned successfully in 95 msec.

SELECT * FROM brancha.project; -- pulls data from the remote DB

Data Output Messages Notifications			
			SQL
	id [PK] integer	name character varying (100)	budget numeric (14,2)
1	1	Road Construction	1000000.00

3. Parallel Query Execution

SHOW max_parallel_workers_per_gather;

SHOW max_parallel_workers;

Data Output Messages Notifications			
			SQL
	max_parallel_workers text		
1	8		

#

SELECT project_id, SUM(amount) AS total_amount

FROM transactions

GROUP BY project_id;

Data Output Messages Notifications			
			SQL
	project_id integer	total_amount numeric	

--Check if a Query Used Parallel Workers

EXPLAIN ANALYZE

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows:

	QUERY PLAN
	text
1	HashAggregate (cost=31.00..33.50 rows=200 width=36) (actual time=0.022..0.023 rows=0.00 loops=1)
2	Group Key: project_id
3	Batches: 1 Memory Usage: 32kB
4	-> Seq Scan on transactions (cost=0.00..24.00 rows=1400 width=22) (actual time=0.019..0.020 rows=0.00 loop...
5	Planning:
6	Buffers: shared hit=1
7	Planning Time: 0.133 ms
8	Execution Time: 0.122 ms

Data Output Messages Notifications

SET

Query returned successfully in 132 msec.

Data Output

Messages

Notifications

≡+

▼

▼

SQL

tx_id [PK] integer	project_id integer	tx_date date	amount numeric (14,2)
-----------------------	-----------------------	-----------------	--------------------------

4. Two-Phase Commit Simulation in PostgreSQL

-- Start transaction in brancha

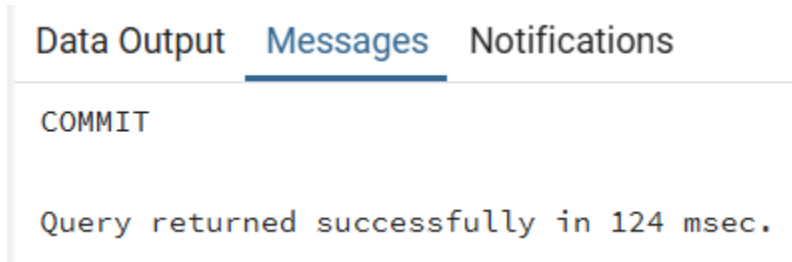
BEGIN;

INSERT INTO brancha.project (name, budget) VALUES ('Airport Expansion', 30000000);

INSERT INTO brancha.expense (project_id, amount, description)

VALUES (1, 5000000, 'Initial Planning Phase');

commit



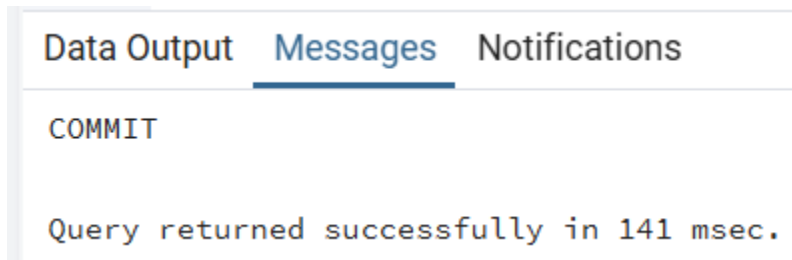
BEGIN;

INSERT INTO branchb.project (name, budget) VALUES ('Bridge Build', 15000000);

INSERT INTO branchb.expense (project_id, amount, description)

VALUES (1, 2000000, 'Materials Purchase');

Commit



5. Distributed Rollback and Recovery (PostgreSQL Simulation)

BEGIN;

```
INSERT INTO brancha.project (name, budget) VALUES ('Power Plant', 100000000);

INSERT INTO brancha.expense (project_id, amount, description) VALUES (1, 10000000, 'Site
Survey');

-- Suppose an error occurs or you need to undo

ROLLBACK;

-- All above changes are undone (not saved)
```

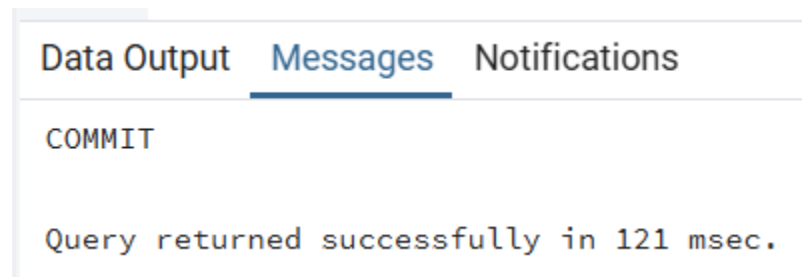
6. Distributed Concurrency Control (PostgreSQL Simulation)

```
BEGIN;

LOCK TABLE brancha.project IN SHARE ROW EXCLUSIVE MODE;

UPDATE brancha.project SET budget = budget + 10000 WHERE id = 2;

COMMIT;
```



```
BEGIN;

SELECT * FROM brancha.project WHERE id = 2 FOR UPDATE;

-- Now only this row is locked and safe for update by this session

UPDATE brancha.project SET budget = budget + 25000 WHERE id = 2;

COMMIT;
```

Data Output Messages Notifications

COMMIT

Query returned successfully in 100 msec.

7. Parallel Data Loading / ETL Simulation in PostgreSQL

```
SELECT project_id, COUNT(*) FROM brancha.transactions GROUP BY project_id;
```

-- PostgreSQL will use parallel workers if configured and the dataset is large.

Data Output	Messages	Notifications
		
		
		
SQL		
project_id integer	count bigint	

8. Three-Tier Client-Server Architecture Design – Three tier architecture

